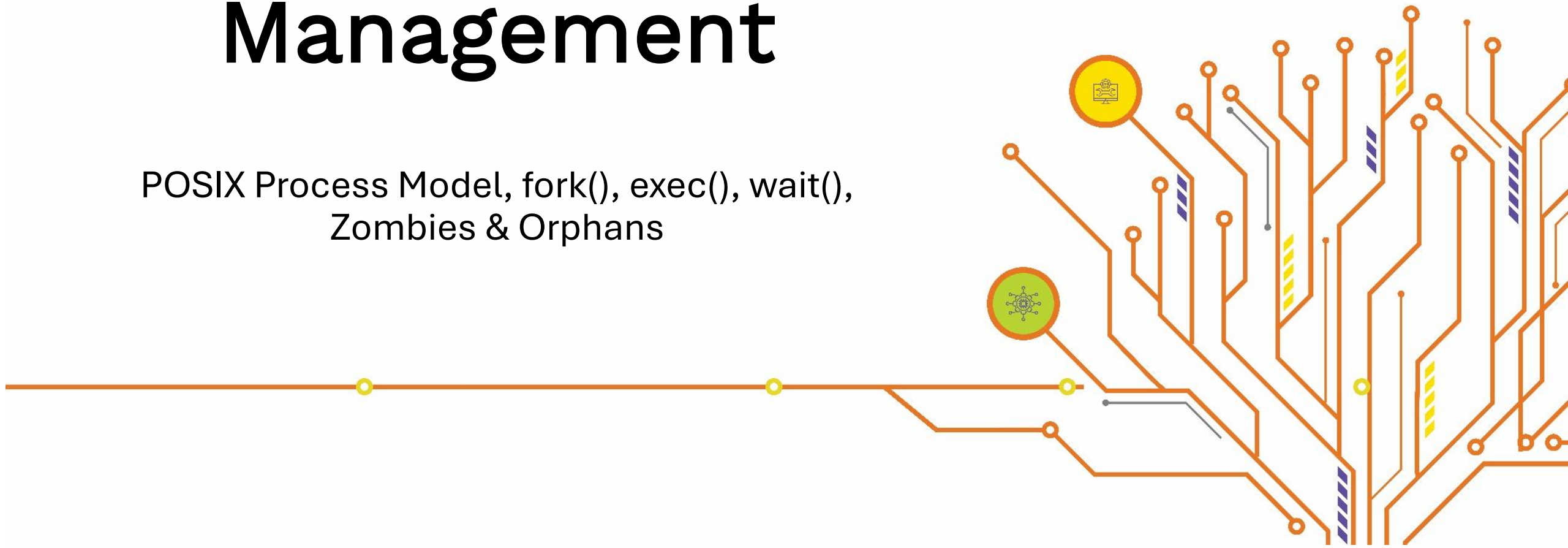
KPIT | APEX LAB

# Process Management

POSIX Process Model, fork(), exec(), wait(), Zombies & Orphans



# Disclaimer

This material has been developed exclusively for educational and academic purposes under the KPIT APEX Lab – Academic CSR Initiative. It is intended strictly for student learning, demonstrations, and classroom use.

Users are advised to verify and confirm the correctness of the contents before use.

It must not be copied, forwarded, shared, reproduced, or redistributed to any other individuals, groups, institutions, or platforms without prior written permission.

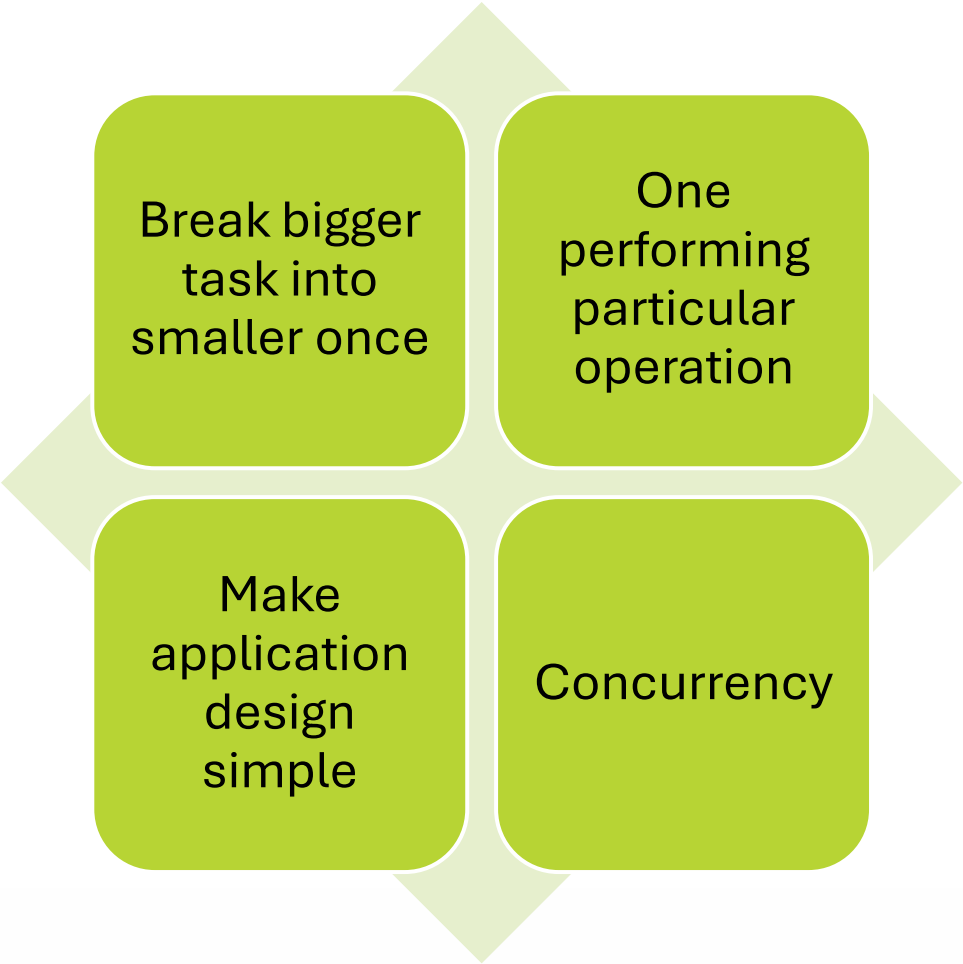
All examples, commands, diagrams, and demonstrations included here are meant solely to support conceptual understanding and learning.

Any use of these contents in commercial products, paid workshops, training programs, or proprietary documentation is strictly prohibited.

# What Is a Process?

- A process is a running instance of a program
- A process has its own virtual address space managed by MMU.
- Contains:
  - Code (text segment)
  - Data & BSS
  - Heap
  - Stack
  - File descriptors
  - Environment variables
  - Registers, PC, SP
- Every process is identified by:
  - PID (Process ID)
  - PPID (Parent PID)
  - Group ID
  - Session ID

# Intension




Examples:

- Logging process
- communication process
- sensor reader process

# Process States

Typical Linux process states:

**R** – Running / Runnable

**S** – Sleeping (interruptible)

**D** – Disk sleep (uninterruptible)

**T** – Stopped

**Z** – Zombie (terminated but parent didn't read exit status)

**X** – Dead (rare; internal kernel use)

Process state can be viewed via /proc/<pid>/status

**Process lifecycle**
new → ready → running → waiting → zombie.

# Parent - Child Processes

Processes have Parent – Child relationship

A process is created by its parent

Processes form a tree rooted at PID 1 (init/systemd)

# Process Management – Core Operations

System programming requires controlling:

- Creation: fork()
- Replacement: exec*()
- Termination: exit()
- Synchronization: wait(), waitpid()
- Information: getpid(), getppid()

# Process ID

- Unique identifier
- Reusable after termination

- Header Required: #include <unistd.h>

- pid_t getpid(void)
- pid_t getppid(void)

# Systems calls related to ID

| Syntax | Return Value / Errors |
|---|---|
| pid_t getpid(void) | Returns calling process PID |
| pid_t getppid(void) | Returns parent PID |
| uid_t getuid(void) | Real user ID |
| uid_t geteuid(void) | Effective user ID |

# Sample Code – Display PID and PPID

- Display Process ID and Parent process ID

```
#include <stdio.h>
#include <unistd.h>

int main(){
        printf("The process id is %d\n", getpid());
        printf("The parent process id is %d\n", getppid());
}
```

# Process Creation - fork()

- Creates a new process running the same image as the current process
- Creates a new PID for child.
- The fork() system call creates a new child process by duplicating the calling (parent) process.

After fork(), both parent and child continue execution independently.

The child gets a copy of the parent's memory, but they do not share stack or heap unless explicitly done via shared memory.

# Sample Code – Process Creation using fork()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main(){
        printf("The process ID is %d\n", getpid());
        pid_t returnValue = fork();
        printf("returnValue = %d\n", returnValue);
        printf("\nAfter fork\n");
}
```

Sample Output

```
The process ID is 1153758
returnValue = 1153899
After fork
returnValue = 0
After fork
```

# Value returned by fork()

pid_t fork(void)

| Process | return value of fork() |
|---|---|
| Parent | Child PID |
| Child | 0 |
| Error | -1 |

# Sample Code

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main(){
	printf("The process ID is %d\n", getpid());
	pid_t returnValue = fork();
	printf("returnValue = %d\n", returnValue);

	if (returnValue == 0) {
		printf("Child process with ID %d\n",getpid());
	} else {
		printf("Parent process with ID %d\n",getpid());
	}

	printf("\nAfter fork\n");
}
```

Sample Output

```
The process ID is 1156041
returnValue = 1156160
Parent process with ID 1156041
After fork
returnValue = 0
Child process with ID 1156160
After fork
```

# Parent and Child Process with global and local data

Parent and child have separate copies of global & local variables.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

static int global_data = 111;

int main(){
	int local_data = 222;
	printf("The process ID is %d\n", getpid());
	pid_t returnValue = fork();
	printf("returnValue = %d\n", returnValue);

	if (returnValue == 0) {
		printf("I'm a Child process with ID %d\n",getpid());
		printf("Value of Global Data = %d and Local Data = %d",global_data,local_data);
		global_data *= 2;
		local_data *= 2;
		printf("Value of Global Data = %d and Local Data = %d",global_data,local_data);
	} else {
		printf("I'm a Parent process with ID %d\n",getpid());
		printf("Value of Global Data = %d and Local Data = %d",global_data,local_data);
	}
	printf("\nAfter fork\n");
}
```

# wait

- Each process has entry in process table
- Used to give information on how the child process exited

- #include<sys/wait.h>

```
pid_t wait(int *status);
```

- Returns **PID** of the terminated child
- Returns -1 on error
- Stores exit information in status (if not NULL)

# What Does wait() Really Do?

- Blocks until any child process exits
- Reaps (removes) that child from the process table
- Returns the child's PID
- Stores exit status and signal info in status
- If the parent never calls wait(), zombie processes accumulate.

# Sample code – wait()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(){

int status = 0;
pid_t returnValue = fork();

if (returnValue == 0) {
        printf("I'm a Child process with ID %d\n",getpid());
        sleep(5);
        printf("Exiting from Child Process.\n");
} else {
        wait(&status);
        printf("I'm a Parent process with ID %d\n",getpid());
}
}
```

Sample Output

```
I'm a Child process with ID 1186735
Exiting from Child Process.
I'm a Parent process with ID 1186574
```

# Why wait() Is Not Enough

- wait() always waits for **any** child.
- If you have multiple children, you cannot choose which one to wait for.
- Also, wait() **blocks** until *some* child exits.
- For more control, POSIX provides: 👉 waitpid()

# waitpid() — the Better Version of wait()

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- Wait for **one specific child**
- Wait for **any child**
- Wait **non-blocking**
- Support job control signals
- Used in almost all real systems

# Sample code – waitpid()

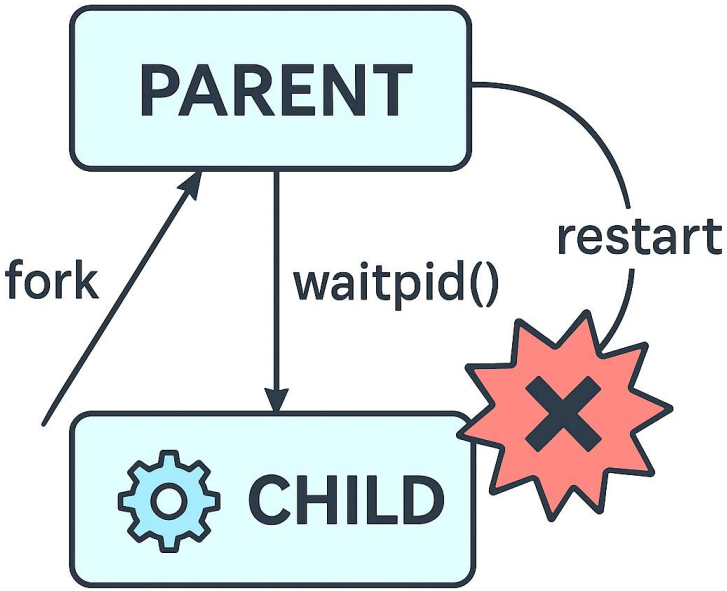
```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(){

int status = 0;
pid_t returnValue = fork();


if (returnValue == 0) {
        printf("I'm a Child process with ID %d\n",getpid());
        sleep(5);
        printf("Exiting from Child Process.\n");
} else {
        waitpid(returnValue, &status, 0);
        printf("I'm a Parent process with ID %d\n",getpid());
}
}
```

Sample Output

```
I'm a Child process with ID 1186735
Exiting from Child Process.
I'm a Parent process with ID 1186574
```

# Scenario 1: Supervisor/Watchdog Restart Loop

- Many embedded services need to **restart automatically** if they crash.
- Parent process forks a child
- Child runs the actual service
- Parent waits using waitpid()
- If the child crashes → parent restarts it



# Orphan Process

- An orphan process is a process whose parent process has terminated, while the child process continues to run.
- The operating system reassigns the orphaned child to init (PID 1) or systemd in Linux systems.
- init/systemd becomes the new parent and is responsible for:
  - Monitoring the orphan process
  - Cleaning up its resources when it finishes
- Orphan processes themselves are not harmful. They are normal and well-handled by the OS.

# Sample Code – Orphan Process

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(){

int status = 0;
pid_t id = fork();

if (id == 0) {
        printf("I'm a Child process with ID %d\n",getpid());
        sleep(10);
        printf("Exiting from Child Process.\n");
} else {
        printf("I'm a Parent process with ID %d\n",getpid());
return 0;
}
}
```

# Zombie Process

- A zombie process (also called a *defunct process*) is:
- A child process that has **already terminated**
- But its **parent has not read its exit status**
- The kernel keeps a small entry for it in the process table
- This leftover entry allows the parent to call wait() and find out:
  - The child's exit code
  - Whether it was terminated normally or by a signal
  - Other info communicated through the exit status

# Sample Code – Zombie Process

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(){

int status = 0;
pid_t id = fork();

if (id == 0) {
        printf("I'm a Child process with PID %d\n",getpid());
        sleep(5);
        printf("Exiting from Child Process.\n");
} else {
        printf("I'm a Parent process with PID %d\n",getpid());
        sleep(15);
        waitpid(id, NULL, 0);
        printf("Exiting from Child Process.\n");
}
}
```

# Why Zombies Can Be a Problem ?

- One zombie is harmless — it uses almost **no memory**.
- But a large number of zombies can cause:
- **PID exhaustion** (you can't create new processes)
- System instability
- Resource leaks
- This usually happens when:
- Parent process never calls wait()

# Executing a New Program — exec()

- Replaces the current process image with a new program, not PID
- exec family: execl, execlp, execv, execve, execvp
- Does not create a new process — PID stays the same
- After success, the old code/data/heap/stack no longer exist
- Used after fork() by the child process to load another program

```
int execv(const char *path, char *const argv[])
```

- path: path to executable;
- argv: NULL-terminated array
- Does not return on success. On error: -1

# Sample Code 1 – execv()

main.c

```
#include <stdio.h>
#include <unistd.h>
#include <errno.h>


int main(){
        printf("This is main.c (PID=%d)\n", getpid());
        char* arg[] = {"newfile", NULL};
        execv("./newfile",arg);
        perror("execv failed");
        printf("This is still main.c because execv() failed.\n");
return 0;
}
```

newfile.c

```
#include <stdio.h>
#include <unistd.h>


int main() {
        printf("Hello from newfile.c! (PID=%d,
        PPID=%d)\n", getpid(), getppid());
        return 0;
}
```

# Sample Code 2 – execv()

Passing arguments to new program

main.c

```c
#include <stdio.h>
#include <unistd.h>
#include <errno.h>

int main(){
	printf("This is main.c (PID=%d)\n", getpid());
	char* arg[] = {"newfile", "alpha", "Hello There!", NULL};
	execv("./newfile",arg);

	perror("execv failed");
	printf("This is still main.c because execv() failed.\n");
	return 0;
}
```

newfile.c

```c
#include <stdio.h>
#include <unistd.h>

int main(int argc, char* argv[]) {
	printf("Hello from newfile.c! (PID=%d,
	PPID=%d)\n", getpid(), getppid());

	printf("argc = %d\n", argc);
	for (int i = 0; i < argc; i++) {
	printf("argv[%d] = '%s'\n", i, argv[i]);
	}
	return 0;
}
```

# Scenario - ECU Config Snapshot Backup

Context:

During vehicle diagnostics, a tool exports the current ECU configuration snapshot to a text file (e.g., parameter settings, feature flags, VIN, timestamps). Before running any experiments or flashing firmware, engineers must create a backup copy of this snapshot so they can restore the exact state if needed.

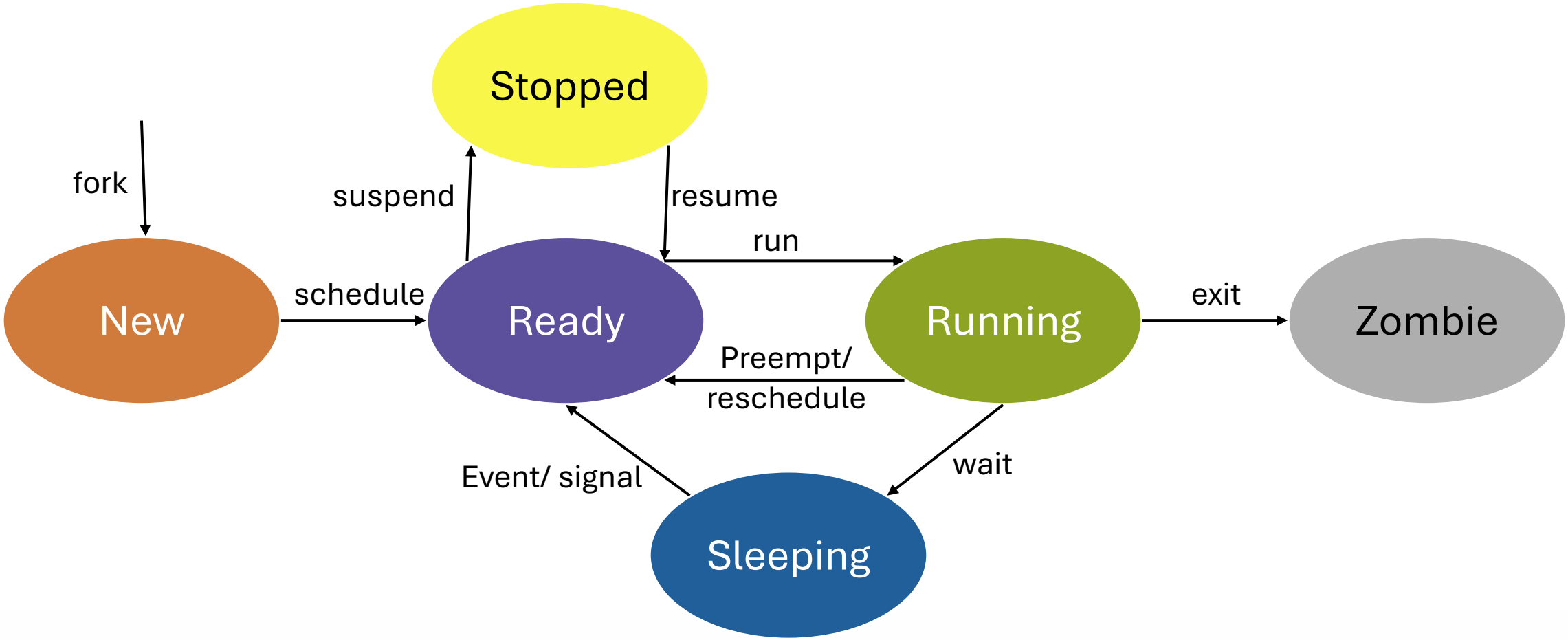
What tiny utility does:

- The parent process reads the entire original snapshot ecu_config_snapshot.txt.
- It then forks one child.
- The child writes that exact content into backup_ecu_config.txt.
- The parent prints a message (“Child is creating a copy…”) and waits for the child.

# Summary

- fork creates
- exec replaces
- wait reaps
- zombies = un-reaped children
- orphans adopted by PID 1

# Process Lifecycle



# Summary: fork() Memory Diagram

Before fork()

**Parent Process (PID = 1200)**

Text | Data | BSS | Heap | Stack | FD Table

fork();
After fork()

| **Parent Process (PID = 1200)** | | **Child Process (PID = 1201)** |
|---|---|---|
| Text \| Data \| BSS → | Shared (Copy-on-Write)separate → | Text \| Data \| BSS |
| Heap → | copies after first write → | Heap |
| Stack → | separate → | Stack |
| FD Table → | points to same Open File Table entries → | FD Table |

# Summary: exec()

Before exec()

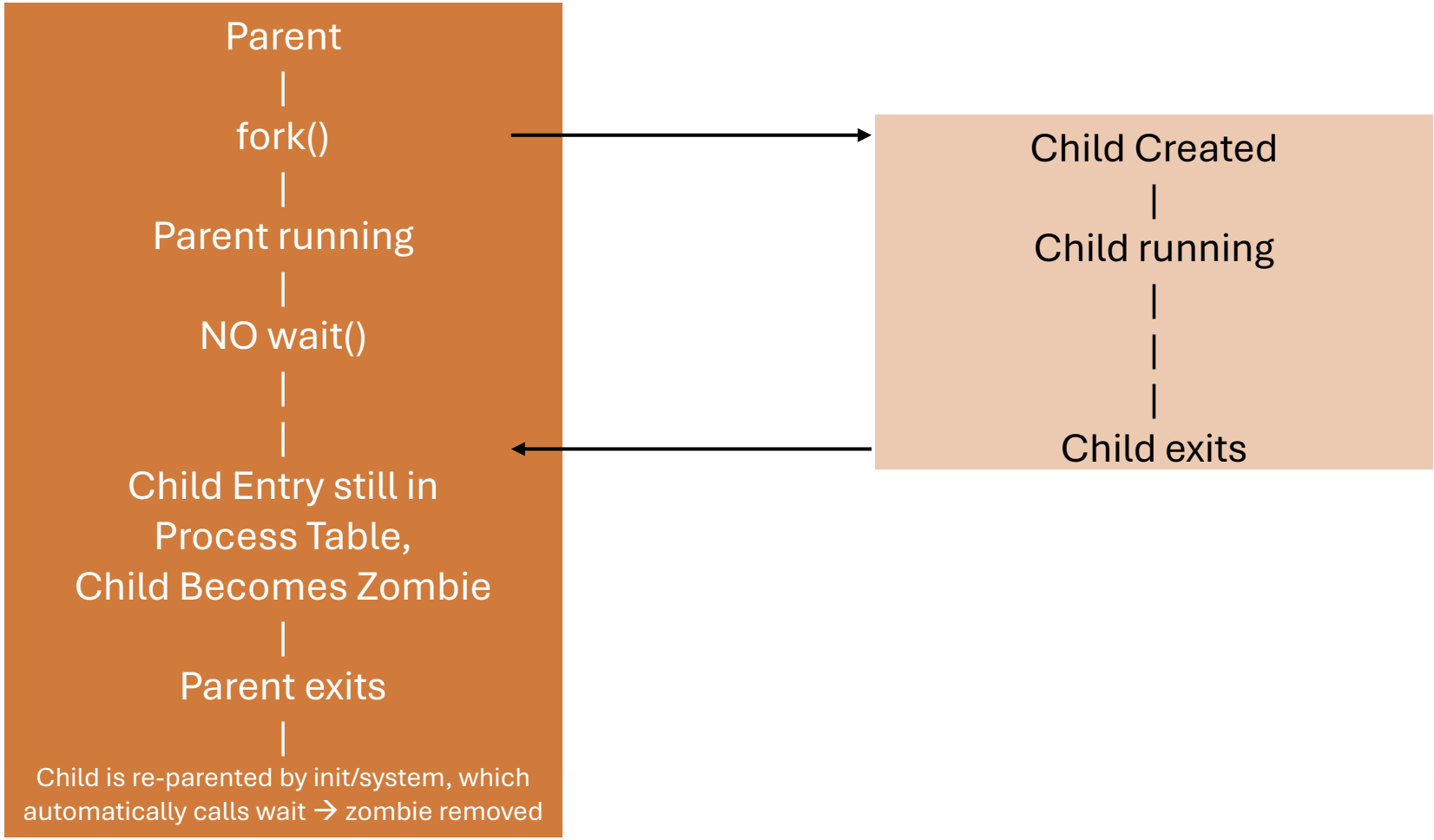
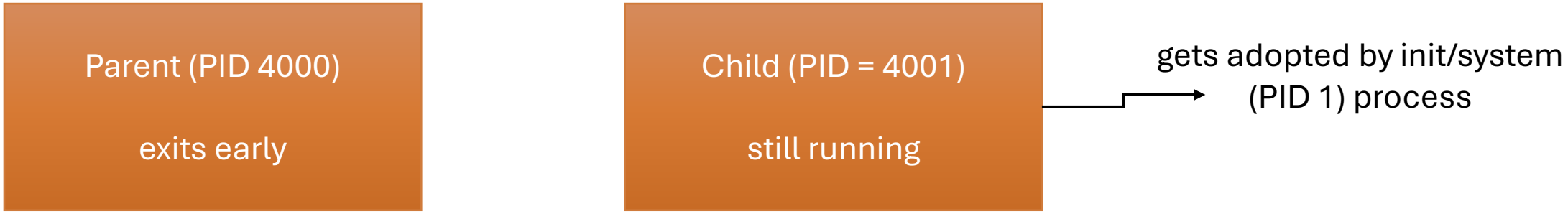
Process (PID = 2345)
Running program: main.c
Memory: text/data/bss/heap/stack for main.c
FDs remain open

exec("./newfile")

After exec()

Same Process (PID = 2345)
Now Running program: newfile
Memory replaced entirely with new program
Only FDs survive (if not marked O_CLOEXEC)

# Summary: Zombie Process



# Summary: Orphan Process